ITES
RENÉ DESCARTES

Carrera:
Licenciatura en programación y web máster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
Arquitectura de Software

Alumno(s):
Jose Enrique Ramirez Navarro

1. Introducción y método

Este trabajo compara cinco frameworks de PHP con los mismos diez criterios, para ver en qué se parecen, en qué se diferencian y en qué tipo de proyecto conviene cada uno. Laravel es la referencia obligatoria del curso; los otros cuatro se eligieron para cubrir perfiles distintos: Symfony (enfoque empresarial y modular), CodeIgniter (ligero y sencillo), CakePHP (basado en convenciones) y Slim (microframework).

Para ubicar la comparación conviene distinguir cuatro conceptos. Un **framework** aporta una estructura completa de proyecto y reglas para organizar el código. Una **biblioteca** (por ejemplo Doctrine o Twig) resuelve una tarea concreta y es el programador quien la invoca. Un **CMS** (WordPress, Drupal, Joomla) es una aplicación ya construida que se configura y se extiende. Un **microframework** es un framework mínimo que se limita a rutas y manejo de peticiones HTTP; Slim es el ejemplo incluido aquí.

Todas las consultas se hicieron el 3 de noviembre de 2026. Por versión estable se entiende la última publicación sin etiquetas de prueba (RC o beta). Los datos de versión, fecha, requisitos y soporte se tomaron de documentación oficial, de los repositorios de GitHub de cada proyecto y de Packagist; la última fila del cuadro indica las fuentes de cada columna, y la lista completa está en las referencias. Los datos de cantidad de ofertas laborales no se midieron con cifras, por lo que la demanda laboral se describe de forma cualitativa.

2. Cuadro comparativo

El cuadro aparece en las siguientes páginas, con los frameworks en columnas y los criterios en filas.

Cuadro 1. Comparación de cinco frameworks de PHP (consulta: 3 de noviembre de 2026).

Criterio	Laravel	Symfony	CodeIgniter	CakePHP	Slim
1. Identificación y vigencia	**Laravel 13** (v13.40.0, publicada el 3 nov 2026; la serie 13 salió el 17 mar 2026). Consulta: 3 nov 2026. **PHP mínimo:** 8.3.	**Symfony 8.1** (rama estable de mayo de 2026; parche 8.1.10 según symfony.com). Consulta: 3 nov 2026. **PHP mínimo:** 8.4 (la LTS 7.4 pide 8.2).	**CodeIgniter 4.7.6** (publicada el 1 nov 2026). Consulta: 3 nov 2026. **PHP mínimo:** 8.2.	**CakePHP 5.4** (5.4.0 del 19 jul 2026; último parche visto: 5.4.4). Consulta: 3 nov 2026. **PHP mínimo:** 8.2.	**Slim 4.15** (4.15.1 del 21 nov 2025; GitHub lista 4.15.3 como la más reciente). Consulta: 3 nov 2026. **PHP mínimo:** 7.4 en composer.json; en la práctica conviene 8.2 o superior.
2. Licencia y costo	Licencia MIT. Uso comercial libre y sin costo. Solo son de pago servicios opcionales de la empresa (hosting, paneles), no el framework.	Licencia MIT. Uso comercial libre y sin costo. El soporte profesional de terceros es opcional.	Licencia MIT. Uso comercial libre y sin costo.	Licencia MIT. Uso comercial libre y sin costo.	Licencia MIT. Uso comercial libre y sin costo.
3. Arquitectura	MVC con contenedor de inyección de dependencias y *pipeline* de middleware. Organiza el proyecto en app/, routes/ y resources/views.	Basado en componentes desacoplados y un núcleo dirigido por eventos (HttpKernel). Se organiza en src/, config/ y templates/.	MVC ligero con espacios de nombres. Separa app/ y public/; admite módulos y filtros (middleware).	MVC con «convención sobre configuración»: los nombres de clases y tablas definen el comportamiento. Usa middleware PSR-15.	No impone MVC. Funciona como *pipeline* de middleware PSR-15 con rutas; el equipo decide la estructura de carpetas.
4. Componentes incluidos	**ORM:** Eloquent, incluido; define modelos y relaciones sin SQL manual. **Plantillas:** Blade. **Auth:** integrada, con kits de inicio (React, Vue, Svelte, Livewire). **Migraciones, colas, correo y pruebas:** incluidos. **CLI:** Artisan.	**ORM:** no incluye uno propio; se integra con Doctrine, que se instala aparte. **Plantillas:** Twig (paquete aparte). **Auth:** componente Security. **Migraciones:** Doctrine Migrations. **Colas:** Messenger. **CLI:** bin/console.	**ORM:** sin ORM completo; ofrece Model con Query Builder y Entities. **Plantillas:** View Parser y vistas PHP. **Auth:** CodeIgniter Shield, paquete oficial aparte. **Migraciones y pruebas:** incluidas. **Colas:** fuera del núcleo. **CLI:** spark.	**ORM:** propio e incluido (Table y Entity). **Plantillas:** vistas PHP con helpers. **Auth:** plugins oficiales Authentication y Authorization (aparte). **Migraciones y colas:** plugins Migrations y Queue. **CLI:** bin/cake y generador Bake.	**Ninguno de serie:** no trae ORM, plantillas, autenticación, migraciones, colas ni consola. Se agregan con Composer; por ejemplo, Twig-View para plantillas y PHPUnit para pruebas.

Criterio	Laravel	Symfony	CodeIgniter	CakePHP	Slim
5. Dependencias y ecosistema	Usa Composer. Paquetes oficiales como Sanctum, Horizon, Socialite y Cashier; gran cantidad de paquetes de terceros en Packagist.	Usa Composer y Symfony Flex. Sus componentes se pueden usar por separado y los emplean otros proyectos; hay bundles de terceros.	Composer es la vía recomendada. Paquetes oficiales (Shield, Settings); ecosistema de terceros más pequeño.	Usa Composer. Plugins oficiales (Authentication, Migrations, DebugKit, Bake); comunidad de plugins más pequeña.	Usa Composer. Depende de estándares PSR y de FastRoute; el equipo elige la implementación PSR-7 y el contenedor (PHP-DI recomendado).
6. Curva de aprendizaje y documentación	Curva baja a moderada. Documentación oficial completa y actualizada por versión, en inglés, con muchos ejemplos; abundan cursos y comunidad hispana.	Curva alta: exige entender inyección de dependencias y configuración. Documentación oficial extensa y mantenida, en inglés.	Curva baja. Guía de usuario clara, en inglés, con ejemplos breves; es fácil comenzar.	Curva media: hay que aprender las convenciones. Documentación oficial (Cookbook) en inglés, con ejemplos y tutoriales.	Arranque muy rápido, pero la curva sube al integrar piezas. Documentación breve en inglés.
7. Rendimiento y complejidad	Framework completo. Más capas que uno ligero, lo que añade algo de costo; se compensa con caché y colas. Configuración inicial sencilla (.env).	Completo pero modular: carga solo los componentes que se usan. Más archivos de configuración que otros.	Ligero: arranque rápido, pocas dependencias y poca configuración.	Completo de tamaño medio. Las convenciones reducen la configuración; apto para aplicaciones de gestión.	Muy ligero y con mínima sobrecarga, pero cada pieza adicional obliga a configurar e integrar.
8. Comunidad y soporte	Repositorio muy activo: versiones casi semanales (13.34 a 13.40 entre el 29 sep y el 3 nov 2026). Mayor anual; correcciones 18 meses y seguridad 2 años (la 13: hasta Q3 2027 y Q1 2028). Sin LTS. Demanda laboral alta (observación cualitativa).	Versión menor cada 6 meses y LTS cada 2 años. La 7.4 LTS recibe correcciones hasta nov 2028 y seguridad hasta nov 2029; la 8.1 hasta ene 2027. Muy usado en empresas.	Siete versiones 4.7.x entre el 1 feb y el 1 nov 2026. Desarrollo voluntario y sin LTS formal. Demanda laboral menor, ligada a sistemas existentes (cualitativo).	Menores cada 6 a 12 meses; la 5.4 tiene seguridad hasta que salga la 5.7. Comunidad pequeña y sin LTS formal. Demanda laboral baja (cualitativo).	Pocas versiones (4.15.0 en ago 2025 y 4.15.1 en nov 2025): mantenimiento lento. Consecuencia: el equipo depende más de terceros y de su propio criterio.
9. Casos de uso típicos	Aplicaciones web completas, APIs, paneles administrativos y SaaS.	Sistemas empresariales grandes, plataformas de larga vida y APIs complejas.	Aplicaciones pequeñas o medianas y APIs sencillas; funciona bien en hosting básico.	Sistemas de gestión, intranets y aplicaciones CRUD de desarrollo rápido.	APIs REST, microservicios y prototipos.

Criterio	Laravel	Symfony	CodeIgniter	CakePHP	Slim
10. Fortalezas y limitaciones	**Fortalezas:** casi todo integrado; documentación y ecosistema; alta productividad. **Limitaciones:** sin LTS, requiere actualizar cada año; las facades ocultan parte del funcionamiento.	**Fortalezas:** componentes reutilizables y estables; LTS larga; escala bien. **Limitaciones:** curva alta; mucha configuración; la 8.x exige PHP 8.4.	**Fortalezas:** ligero, simple, fácil de hospedar. **Limitaciones:** menos herramientas en el núcleo (sin ORM completo ni autenticación); comunidad menor.	**Fortalezas:** Bake acelera el CRUD; ORM incluido; convenciones claras. **Limitaciones:** menos paquetes de terceros; las convenciones estorban si el diseño se sale de ellas.	**Fortalezas:** mínimo, rápido y flexible; basado en estándares PSR. **Limitaciones:** hay que elegir y mantener cada pieza; ritmo de versiones lento.
Fuentes del dato	(Laravel, 2026a, 2026b, s. f.)	(Symfony, 2026a, 2026b, s. f.)	(CodeIgniter, 2026a, 2026b, s. f.)	(CakePHP, 2026a, 2026b, s. f.)	(Slim, 2026a, 2026b, s. f.)

3. Recomendación para el escenario

Escenario: una clínica pequeña necesita agendar citas, registrar pacientes y enviar recordatorios por correo; en dos años crecerá a varias sucursales y lo mantendrán dos desarrolladores con conocimientos básicos de PHP.

Elegiría Laravel. El criterio 4 (componentes incluidos) pesa mucho: Laravel trae Eloquent para relacionar pacientes, citas y sucursales, además de correo, colas, programador de tareas y autenticación, que son justo las piezas de los recordatorios y del acceso del personal. El criterio 6 (curva de aprendizaje y documentación) también favorece a Laravel: dos desarrolladores con nivel básico pueden apoyarse en una documentación completa y en kits de inicio, y ya es el framework que se trabaja en el curso. El criterio 8 (comunidad y soporte) respalda el crecimiento: la serie 13 recibe correcciones hasta el tercer trimestre de 2027 y seguridad hasta el primero de 2028, y el repositorio publica versiones casi cada semana.

Descartaría Slim. Al ser un microframework no trae ORM, autenticación, migraciones ni colas (criterio 4), así que los dos desarrolladores tendrían que elegir, integrar y mantener cada pieza; ese trabajo pesa más en un equipo pequeño, y el ritmo lento de versiones (criterio 8) deja más responsabilidad en el equipo. Symfony sería una opción válida, pero su curva más alta y su mayor configuración (criterios 6 y 7) no se justifican para este equipo.

Riesgo de la elección: Laravel no tiene versiones LTS y publica una versión mayor cada año, con correcciones durante 18 meses. Si el equipo no planea las actualizaciones, el sistema podría quedar sin parches mientras guarda datos sensibles de pacientes. Además, la serie 13 exige PHP 8.3, por lo que el hosting debe cumplirlo. Se mitiga con un calendario anual de actualización y pruebas automáticas.

4. Conclusión

Al comparar los frameworks aprendí que la pregunta no es cuál es “el mejor”, sino cuánto trae resuelto cada uno y cuánto debe resolver el equipo. Laravel y Symfony ofrecen mucho de fábrica, pero con costos distintos: Laravel se aprende más rápido, mientras que Symfony pide más configuración a cambio de estabilidad a largo plazo. Slim está en el otro extremo: arranca con pocas líneas, pero cada pieza extra queda a cargo de quien programa. También vi que revisar requisitos importa, porque Symfony 8.1 pide PHP 8.4 y Laravel 13 pide PHP 8.3, y eso condiciona el servidor. Para el proyecto del curso, Laravel sigue siendo una buena opción porque cubre ORM, plantillas, autenticación y migraciones con documentación clara. El que más me sorprendió fue CodeIgniter 4: pensaba que era de otra época, pero publicó al menos siete versiones en 2026 y sigue siendo muy ligero, aunque con menos herramientas integradas.

5. Nota sobre el uso de inteligencia artificial

Usé Claude (Anthropic) como apoyo para contrastar información y para preparar un primer borrador del documento. Los datos de versión, fecha, requisitos de PHP, licencia y soporte se contrastaron con las fuentes oficiales citadas en las referencias, y revisé personalmente el contenido del cuadro antes de entregarlo.

Referencias

CakePHP. (s. f.-a). *CakePHP 5 Cookbook*. Recuperado el 3 de noviembre de 2026, de https://book.cakephp.org/5/en/index.html

CakePHP. (2026a). *Releases · cakephp/cakephp* [Repositorio]. GitHub. Recuperado el 3 de noviembre de 2026, de https://github.com/cakephp/cakephp/releases

CakePHP. (2026b). *Home · cakephp/cakephp Wiki* [Mapa de versiones y versiones con soporte]. GitHub. Recuperado el 3 de noviembre de 2026, de https://github.com/cakephp/cakephp/wiki

CodeIgniter. (s. f.). *CodeIgniter 4 user guide*. Recuperado el 3 de noviembre de 2026, de https://codeigniter.com/user_guide/

CodeIgniter. (2026a). *Server requirements* (4.7.6). CodeIgniter 4 User Guide. Recuperado el 3 de noviembre de 2026, de https://codeigniter.com/user_guide/intro/requirements.html

CodeIgniter. (2026b). *Releases · codeigniter4/framework* [Repositorio]. GitHub. Recuperado el 3 de noviembre de 2026, de https://github.com/codeigniter4/framework/releases

Laravel. (s. f.). *Laravel documentation*. Recuperado el 3 de noviembre de 2026, de https://laravel.com/docs

Laravel. (2026a). *Release notes* (13.x). Recuperado el 3 de noviembre de 2026, de https://laravel.com/docs/13.x/releases

Laravel. (2026b). *Releases · laravel/framework* [Repositorio]. GitHub. Recuperado el 3 de noviembre de 2026, de https://github.com/laravel/framework/releases

Slim Framework. (s. f.). *Slim 4 documentation*. Recuperado el 3 de noviembre de 2026, de https://www.slimframework.com/docs/v4/

Slim Framework. (2026a). *slim/slim* [Paquete]. Packagist. Recuperado el 3 de noviembre de 2026, de https://packagist.org/packages/slim/slim

Slim Framework. (2026b). *Releases · slimphp/Slim* [Repositorio]. GitHub. Recuperado el 3 de noviembre de 2026, de https://github.com/slimphp/Slim/releases

Symfony. (s. f.). *Symfony documentation*. Recuperado el 3 de noviembre de 2026, de https://symfony.com/doc

Symfony. (2026a). *Symfony releases*. Recuperado el 3 de noviembre de 2026, de https://symfony.com/releases

Symfony. (2026b). *Symfony 8.1 release*. Recuperado el 3 de noviembre de 2026, de https://symfony.com/releases/8.1